Object-Oriented Programming in Java

Core concepts, syntax and worked examples

#	Topic
1	Classes and objects
2	Constructors and object creation
3	Encapsulation and access modifiers
4	Inheritance
5	Polymorphism: overloading and overriding
6	Abstraction: abstract classes and interfaces
7	static, final and the Object class
8	Composition vs inheritance
9	A worked example: a small banking model
10	Quick revision checklist

1. Classes and objects

A class is a blueprint that describes state (fields) and behaviour (methods). An object is a concrete instance of that blueprint, living on the heap and reached through a reference variable. Java is class-based: every piece of code you write belongs to some class.

```
public class Student {
    // fields - the state of each object
    private String name;
    private int rollNo;

    // method - the behaviour
    public String describe() {
        return rollNo + " - " + name;
    }
}

Student s = new Student();   // s is a reference; the object is on the heap
```

Members of a class

- **Fields** (instance variables): one copy per object.
- **Methods**: operations, may read and change the object's state.
- **Constructors**: run once, when the object is created.
- **Initializer blocks**: `{ }` runs before every constructor; `static { }` runs once when the class is loaded.
- **Nested classes**: static nested, inner, local and anonymous classes.

> Note: `Student s;` alone creates no object. Until you assign it, `s` is `null`, and calling a method on it throws `NullPointerException`.

2. Constructors and object creation

A constructor has the same name as the class and no return type. If you write no constructor at all, the compiler supplies a no-argument default one; the moment you write any constructor, that free default disappears.

```
public class Student {
    private final String name;
    private int rollNo;

    public Student(String name) {          // constructor
        this.name = name;                  // 'this' disambiguates field vs parameter
    }

    public Student(String name, int rollNo) {
        this(name);                        // constructor chaining - must be first statement
        this.rollNo = rollNo;
    }
}
```

Order of initialization when `new` runs

- Static fields and static blocks of the class (once, at class load).
- Memory allocated, fields set to defaults (0, false, null).
- Superclass constructor via `super(...)`, implicit if not written.
- Instance fields and instance initializer blocks, in source order.

- The body of the constructor itself.

3. Encapsulation and access modifiers

Encapsulation means hiding internal representation behind a controlled interface. Keep fields private, expose only the operations that make sense, and validate inside the setters so an object can never reach an invalid state.

```
public class Account {
    private double balance;               // hidden state

    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
        balance += amount;                // invariant protected here
    }
}
```

Modifier	Same class	Same package	Subclass (other package)	Anywhere
private	Yes	No	No	No
(default)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

> A getter that returns a mutable object (a list, an array, a `Date`) leaks the internals. Return an unmodifiable view or a defensive copy instead.

4. Inheritance

`extends` creates an is-a relationship: the subclass inherits the accessible members of the superclass and may add or redefine behaviour. Java allows single inheritance of classes but a class may implement many interfaces.

```
class Person {
    protected String name;
    Person(String name) { this.name = name; }
    public String describe() { return "Person: " + name; }
}

class Student extends Person {
    private String course;

    Student(String name, String course) {
        super(name);                      // must be the first statement
        this.course = course;
    }

    @Override
    public String describe() {
        return super.describe() + ", studying " + course;
    }
}
```

- Constructors are not inherited; private members are not accessible in the subclass.
- `final class` cannot be extended; `final` methods cannot be overridden.
- Every class implicitly extends `java.lang.Object`.

5. Polymorphism

Polymorphism lets one reference type work with many runtime types. Java has two flavours: compile-time (overloading) and runtime (overriding, resolved by dynamic dispatch).

Overloading vs overriding

	Overloading	Overriding
Where	Same class (or inherited)	Subclass redefines a superclass method
Signature	Name same, parameters differ	Name and parameters identical
Return type	May differ freely	Same, or a covariant subtype
Access	May differ freely	Cannot be more restrictive
Resolved	At compile time, by the static type	At runtime, by the actual object
Static methods	Can be overloaded	Hidden, not overridden

```
Person p = new Student("Asha", "Physics");   // static type Person, runtime type Student
System.out.println(p.describe());            // calls Student.describe() - dynamic dispatch

// p.getCourse();        // compile error: Person has no such method
if (p instanceof Student s) {                // pattern matching (Java 16+)
    System.out.println(s.getCourse());
}
```

> Always mark overrides with `@Override`. It costs nothing and turns a silent typo (a new unused method) into a compile error.

6. Abstraction

Abstraction exposes what an object does while hiding how. In Java it is expressed with abstract classes and interfaces.

```
abstract class Shape {
    abstract double area();                      // no body - subclasses must supply one
    public String label() { return getClass().getSimpleName() + " area=" + area(); }
}

interface Drawable {
    int MAX_LAYERS = 8;                          // implicitly public static final
    void draw();                                 // implicitly public abstract
    default void redraw() { draw(); }            // default method (Java 8+)
    static Drawable nothing() { return () -> {}; }
}

class Circle extends Shape implements Drawable {
    private final double r;
    Circle(double r) { this.r = r; }
    @Override double area() { return Math.PI * r * r; }
    @Override public void draw() { System.out.println("circle r=" + r); }
}
```

	Abstract class	Interface
State	Can hold instance fields	Only public static final constants
Constructors	Yes	No

Methods	Abstract and concrete	abstract, default, static, private
Multiple use	One superclass only	Any number implemented
Use it for	Shared code among close relatives	A capability many unrelated types can offer

7. static, final and the Object class

`static` members belong to the class rather than to any instance, so a static method has no `this` and cannot touch instance fields directly. `final` means assign once: a final field cannot be reassigned, a final method cannot be overridden, a final class cannot be extended.

```
class Counter {
    private static int created = 0;       // one copy shared by all instances
    private final int id;                 // assigned once, in the constructor

    Counter() { id = ++created; }

    static int getCreated() { return created; }
}
```

Methods worth overriding from Object

- **equals(Object)** - logical equality. Must be reflexive, symmetric, transitive and consistent.
- **hashCode()** - always override it together with equals, or hash-based collections break.
- **toString()** - a readable description; invaluable in logs and debuggers.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Student other)) return false;
    return rollNo == other.rollNo && Objects.equals(name, other.name);
}

@Override
public int hashCode() { return Objects.hash(name, rollNo); }
```

> For a plain immutable data carrier, a `record` (Java 16+) generates the constructor, accessors, equals, hashCode and toString for you: `record Point(int x, int y) { }`.

8. Composition vs inheritance

Inheritance models is-a; composition models has-a. Inheritance is the tighter coupling, because the subclass depends on the superclass's implementation details and a change upstream can break it silently. The usual guidance is to prefer composition and reach for inheritance only when the subtype genuinely is a substitutable kind of the supertype.

```
// Inheritance - a Car IS-A Vehicle
class Car extends Vehicle { }

// Composition - a Car HAS-AN Engine
class Car {
    private final Engine engine;
    Car(Engine engine) { this.engine = engine; }   // dependency injected, easy to swap or mo
ck
    void start() { engine.ignite(); }
}
```

> Liskov substitution: anywhere the supertype is expected, any subtype must work without surprising the caller. If overriding forces you to weaken a guarantee or throw `UnsupportedOperationException`, the hierarchy is wrong.

9. Worked example: a small banking model

This pulls the pieces together: an abstract base holding shared state, an interface for an optional capability, two subclasses overriding behaviour, and polymorphic use at the call site.

```
interface InterestBearing {
    double monthlyRate();
    default double interestFor(double balance) { return balance * monthlyRate(); }
}

abstract class Account {
    private final String id;
    private double balance;

    protected Account(String id, double opening) {
        this.id = id;
        this.balance = opening;
    }

    public String getId()      { return id; }
    public double getBalance() { return balance; }

    public void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("deposit must be positive");
        balance += amt;
    }

    public void withdraw(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("withdrawal must be positive");
        if (amt > balance - minimumBalance())
            throw new IllegalStateException("insufficient funds in " + id);
        balance -= amt;
    }

    protected abstract double minimumBalance();   // each kind decides its own floor

    @Override
    public String toString() {
        return getClass().getSimpleName() + "[" + id + ", " + balance + "]";
    }
}

class SavingsAccount extends Account implements InterestBearing {
    SavingsAccount(String id, double opening) { super(id, opening); }
    @Override protected double minimumBalance() { return 500; }
    @Override public double monthlyRate()       { return 0.004; }
}

class CurrentAccount extends Account {
    private final double overdraft;
    CurrentAccount(String id, double opening, double overdraft) {
        super(id, opening);
        this.overdraft = overdraft;
    }
    @Override protected double minimumBalance() { return -overdraft; }
}

public class Bank {
    public static void main(String[] args) {
        List<Account> accounts = List.of(
            new SavingsAccount("SB-1", 5_000),
            new CurrentAccount("CA-1", 2_000, 10_000));
```

```
        for (Account a : accounts) {
            a.deposit(1_000);
            if (a instanceof InterestBearing ib)      // capability check, not type check
                a.deposit(ib.interestFor(a.getBalance()));
            System.out.println(a);                    // toString() dispatched dynamically
        }
    }
}
```

10. Quick revision checklist

The four pillars

- **Encapsulation** - private fields, controlled access, invariants enforced in one place.
- **Inheritance** - reuse and specialise an existing type through `extends`.
- **Polymorphism** - one interface, many implementations; resolved at runtime.
- **Abstraction** - publish the contract, hide the mechanism.

Points examiners and interviewers press on

- Java has no multiple inheritance of classes; interfaces with default methods give you multiple inheritance of behaviour but never of state.
- Overloading is resolved from the compile-time type; overriding from the runtime type.
- Static and private methods are not polymorphic; a static method in a subclass hides rather than overrides.
- An overriding method may not narrow access nor add new checked exceptions.
- `equals` and `hashCode` travel together.
- `final` on a reference freezes the reference, not the object it points to.
- Never call an overridable method from a constructor: the subclass part is not initialised yet.

Common design principles

- **Single responsibility** - one reason to change per class.
- **Open/closed** - extend behaviour without editing existing, tested code.
- **Liskov substitution** - subtypes must be usable wherever the supertype is.
- **Interface segregation** - many small contracts beat one fat one.
- **Dependency inversion** - depend on abstractions, pass collaborators in.